

An eigenface (EYE-gən-) is the name given to a set of eigenvectors when used in the computer vision problem of human face recognition. The approach of using eigenfaces for recognition was developed by Sirovich and Kirby and used by Matthew Turk and Alex Pentland in face classification. The eigenvectors are derived from the covariance matrix of the probability distribution over the high-dimensional vector space of face images. The eigenfaces themselves form a basis set of all images used to construct the covariance matrix. This produces dimension reduction by allowing the smaller set of basis images to represent the original training images. Classification can be achieved by comparing how faces are represented by the basis set.

History

The eigenface approach began with a search for a low-dimensional representation of face images. Sirovich and Kirby showed that principal component analysis could be used on a collection of face images to form a set of basis features. These basis images, known as eigenpictures, could be linearly combined to reconstruct images in the original training set. If the training set consists of M images, principal component analysis could form a basis set of N images, where $N < M$. The reconstruction error is reduced by increasing the number of eigenpictures; however, the number needed is always chosen less than M . For example, if you need to generate a number of N eigenfaces for a training set of M face images, you can say that each face image can be made up of "proportions" of all the K "features" or eigenfaces:

$$\text{Face image}_1 = (23\% \text{ of } E_1) + (2\% \text{ of } E_2) + (51\% \text{ of } E_3) + \dots + (1\% \text{ of } E_N).$$

In 1991 M. Turk and A. Pentland expanded these results and presented the eigenface method of face recognition. In addition to designing a system for automated face recognition using eigenfaces, they showed a way of calculating the eigenvectors of a covariance matrix such that computers of the time could perform eigen-decomposition on a large number of face images. Face images usually occupy a high-dimensional space and conventional principal component

analysis was intractable on such data sets. Turk and Pentland's paper demonstrated ways to extract the eigenvectors based on matrices sized by the number of images rather than the number of pixels.

Once established, the eigenface method was expanded to include methods of preprocessing to improve accuracy. Multiple manifold approaches were also used to build sets of eigenfaces for different subjects and different features, such as the eyes.

Generation

A set of eigenfaces can be generated by performing a mathematical process called principal component analysis (PCA) on a large set of images depicting different human faces. Informally, eigenfaces can be considered a set of "standardized face ingredients", derived from statistical analysis of many pictures of faces. Any human face can be considered to be a combination of these standard faces. For example, one's face might be composed of the average face plus 10% from eigenface 1, 55% from eigenface 2, and even -3% from eigenface 3. Remarkably, it does not take many eigenfaces combined together to achieve a fair approximation of most faces. Also, because a person's face is not recorded by a digital photograph, but instead as just a list of values (one value for each eigenface in the database used), much less space is taken for each person's face.

The eigenfaces that are created will appear as light and dark areas that are arranged in a specific pattern. This pattern is how different features of a face are singled out to be evaluated and scored. There will be a pattern to evaluate symmetry, whether there is any style of facial hair, where the hairline is, or an evaluation of the size of the nose or mouth. Other eigenfaces have patterns that are less simple to identify, and the image of the eigenface may look very little like a face.

The technique used in creating eigenfaces and using them for recognition is also used outside of face recognition: handwriting recognition, lip reading, voice recognition, sign

language/hand gestures interpretation and medical imaging analysis. Therefore, some do not use the term eigenface, but prefer to use 'eigenimage'.

Practical implementation

To create a set of eigenfaces, one must:

Prepare a training set of face images. The pictures constituting the training set should have been taken under the same lighting conditions, and must be normalized to have the eyes and mouths aligned across all images. They must also be all resampled to a common pixel resolution ($r \times c$). Each image is treated as one vector, simply by concatenating the rows of pixels in the original image, resulting in a single column with $r \times c$ elements. For this implementation, it is assumed that all images of the training set are stored in a single matrix T , where each column of the matrix is an image.

Subtract the mean. The average image a has to be calculated and then subtracted from each original image in T .

Calculate the eigenvectors and eigenvalues of the covariance matrix S . Each eigenvector has the same dimensionality (number of components) as the original images, and thus can itself be seen as an image. The eigenvectors of this covariance matrix are therefore called eigenfaces. They are the directions in which the images differ from the mean image. Usually this will be a computationally expensive step (if at all possible), but the practical applicability of eigenfaces stems from the possibility to compute the eigenvectors of S efficiently, without ever computing S explicitly, as detailed below.

Choose the principal components. Sort the eigenvalues in descending order and arrange eigenvectors accordingly. The number of principal components k is determined arbitrarily by setting a threshold ϵ on the total variance. Total variance

v

=

(

λ

1

+

λ

2

+

.

.

.

+

λ

n

)

$$v=(\lambda _{1}+\lambda _{2}+...+\lambda _{n})$$

, n = number of components, and

λ

$$\lambda$$

represents component eigenvalue.

k is the smallest number that satisfies

(

λ

1

+

λ

2

+

.

.

.

+

λ

k

)

v

>

€

$$\frac{(\lambda_1 + \lambda_2 + \dots + \lambda_k)}{v} > \epsilon$$

These eigenfaces can now be used to represent both existing and new faces: we can project a new (mean-subtracted) image on the eigenfaces and thereby record how that new face differs from the mean face. The eigenvalues associated with each eigenface represent how much the images in the training set vary from the mean image in that direction. Information is lost by projecting the image on a subset of the eigenvectors, but losses are minimized by keeping those eigenfaces with the largest eigenvalues. For instance, working with a 100×100 image will produce 10,000 eigenvectors. In practical applications, most faces can typically be identified using a projection on between 100 and 150 eigenfaces, so that most of the 10,000 eigenvectors can be discarded.

Matlab example code

Here is an example of calculating eigenfaces with Extended Yale Face Database B. To evade computational and storage bottleneck, the face images are sampled down by a factor $4 \times 4 = 16$.

Note that although the covariance matrix S generates many eigenfaces, only a fraction of those are needed to represent the majority of the faces. For example, to represent 95% of the

total variation of all face images, only the first 43 eigenfaces are needed. To calculate this result, implement the following code:

Computing the eigenvectors

Performing PCA directly on the covariance matrix of the images is often computationally infeasible. If small images are used, say 100×100 pixels, each image is a point in a 10,000-dimensional space and the covariance matrix S is a matrix of $10,000 \times 10,000 = 10^8$ elements. However the rank of the covariance matrix is limited by the number of training examples: if there are N training examples, there will be at most $N - 1$ eigenvectors with non-zero eigenvalues. If the number of training examples is smaller than the dimensionality of the images, the principal components can be computed more easily as follows.

Let T be the matrix of preprocessed training examples, where each column contains one mean-subtracted image. The covariance matrix can then be computed as $S = TTT^T$ and the eigenvector decomposition of S is given by

S

v

i

=

T

T

T

v

i

=

λ

i

v

i

$$\{\displaystyle \mathbf{Sv}_i=\mathbf{T} \mathbf{T} ^T\mathbf{v}_i=\lambda _i\mathbf{v}_i\}$$

However $\mathbf{T}\mathbf{T}$ is a large matrix, and if instead we take the eigenvalue decomposition of

$\mathbf{T}$

T

T

u

i

=

λ

i

u

i

$$\mathbf{T}^T \mathbf{T} \mathbf{u}_i = \lambda_i \mathbf{u}_i$$

then we notice that by pre-multiplying both sides of the equation with $\mathbf{T}$, we obtain

$\mathbf{T}$

$\mathbf{T}$

$\mathbf{T}$

$\mathbf{T}$

u

i

=

λ

i

T

u

i

$$\mathbf{T}^T \mathbf{T} \mathbf{u}_i = \lambda_i \mathbf{T} \mathbf{u}_i$$

Meaning that, if $\mathbf{u}_i$ is an eigenvector of $\mathbf{T}^T \mathbf{T}$, then $\mathbf{v}_i = \mathbf{T} \mathbf{u}_i$ is an eigenvector of $\mathbf{S}$. If we have a training set of 300 images of 100×100 pixels, the matrix $\mathbf{T}^T \mathbf{T}$ is a 300×300 matrix, which is much more manageable than the $10,000 \times 10,000$ covariance matrix. Notice however that the resulting vectors $\mathbf{v}_i$ are not normalised; if normalisation is required it should be applied as an extra step.

Connection with SVD

Let $\mathbf{X}$ denote the

d
 $\times$
 n

$$d \times n$$

data matrix with column

x

i

$$\{ \displaystyle x_{i} \}$$

as the image vector with mean subtracted. Then,

c

o

v

a

r

i

a

n

c

e

(

X

)

=

X

X

T

n

$$\mathrm{covariance}(X)=\frac{XX^T}{n}$$

Let the singular value decomposition (SVD) of X be:

X

$=$

U

Σ

V

T

$$X=U{\Sigma }V^{T}$$

Then the eigenvalue decomposition for

X

X

T

$$XX^T$$

is:

X

X

T

=

U

Σ

Σ

Τ

U

Τ

=

U

Λ

U

T

$$X X^T = U \Sigma \Sigma^T U^T = U \Lambda U^T$$

, where $\Lambda = \text{diag}(\text{eigenvalues of } X X^T)$

X

X

T

$$X X^T$$

)

Thus we can see easily that:

The eigenfaces = the first

k

$$\{\displaystyle k\}$$

(

k

$\leq$

n

$$\{\displaystyle k\leq n\}$$

) columns of

U

$$U$$

associated with the nonzero singular values.

The i th eigenvalue of

$$X$$

$$X$$

$$T$$

$$=$$

$$1$$

$$n$$

$$($$

$$XX^T=\frac{1}{n}$$

ith singular value of

X

)

2

$\{\displaystyle X\}^2\}$

(i.e.,

λ

i

=

1

n

σ

i

2

$$\lambda_i = \frac{1}{n} \sum_{j=1}^n \sigma_{ij}^2$$

)

Using SVD on data matrix X , it is unnecessary to calculate the actual covariance matrix to get eigenfaces.

Use in facial recognition

Facial recognition was the motivation for the creation of eigenfaces. For this use, eigenfaces

have advantages over other techniques available, such as the system's speed and efficiency. As eigenface is primarily a dimension reduction method, a system can represent many subjects with a relatively small set of data. As a face-recognition system it is also fairly invariant to large reductions in image sizing; however, it begins to fail considerably when the variation between the seen images and probe image is large.

To recognise faces, gallery images – those seen by the system – are saved as collections of weights describing the contribution each eigenface has to that image. When a new face is presented to the system for classification, its own weights are found by projecting the image onto the collection of eigenfaces. This provides a set of weights describing the probe face. These weights are then classified against all weights in the gallery set to find the closest match. A nearest-neighbour method is a simple approach for finding the Euclidean distance between two vectors, where the minimum can be classified as the closest subject.

Intuitively, the recognition process with the eigenface method is to project query images into the face-space spanned by eigenfaces calculated, and to find the closest match to a face class in that face-space.

Pseudo code

Given input image vector

U

∈

$\mathfrak{R}$

n

$$U \in \mathbb{R}^n$$

, the mean image vector from the database

M

$$M$$

, calculate the weight of the k -th eigenface as:

w

k

=

V

k

T

(

U

-

M

)

$$\{ \displaystyle w_{\{k\}} = V_{\{k\}}^{\{T\}} (U - M) \}$$

Then form a weight vector

W

=

[

w

1

,

w

2

,

.

.

.

,

w

k

,

.

.

.

,

w

n

]

$$\{\displaystyle W=[w_{\{1\}},w_{\{2\}},...,w_{\{k\}},...,w_{\{n\}}]\}$$

Compare W with weight vectors

W

m

$$W_{\{m\}}$$

of images in the database. Find the Euclidean distance.

$$d$$

$$=$$

$$\parallel$$

$$W$$

$$-$$

$$W$$

$$m$$

$$\parallel$$

$$2$$

$$d=\sqrt{W-W_{\{m\}}}$$

If

$$d$$

$$<$$

$$\epsilon$$

$$1$$

$$d<\epsilon_{\{1\}}$$

, then the mth entry in the database is a candidate of recognition.

If

€

1

<

d

<

€

2

$$\{\displaystyle \epsilon_{\{1\}}<d<\epsilon_{\{2\}}\}$$

, then U may be an unknown face and can be added to the database.

If

d

>

ϵ

2

,

U

$$d > \epsilon_{2,U}$$

is not a face image.

The weights of each gallery image only convey information describing that image, not that subject. An image of one subject under frontal lighting may have very different weights to those of the same subject under strong left lighting. This limits the application of such a system. Experiments in the original Eigenface paper presented the following results: an average of 96% with light variation, 85% with orientation variation, and 64% with size variation.

Various extensions have been made to the eigenface method. The eigenfeatures method combines facial metrics (measuring distance between facial features) with the eigenface representation. Fisherface uses linear discriminant analysis and is less sensitive to variation in lighting and pose of the face. Fisherface uses labelled data to retain more of the class-specific information during the dimension reduction stage.

A further alternative to eigenfaces and Fisherfaces is the active appearance model. This approach uses an active shape model to describe the outline of a face. By collecting many face

outlines, principal component analysis can be used to form a basis set of models that encapsulate the variation of different faces.

Many modern approaches still use principal component analysis as a means of dimension reduction or to form basis images for different modes of variation.

Review

Eigenface provides a relatively simple way to realize face recognition in that:

Its training process is completely automatic and easy to code.

Eigenface adequately reduces statistical complexity in face image representation.

Once eigenfaces of a database are calculated, face recognition can be achieved in real time.

Eigenface can handle large databases.

However, the deficiencies of the eigenface method are also obvious:

It is very sensitive to lighting, scale and translation, and requires a highly controlled environment.

Eigenface has difficulty capturing expression changes.

The most significant eigenfaces are mainly about illumination encoding and do not provide useful information regarding the actual face.

To cope with illumination distraction in practice, the eigenface method usually discards the first three eigenfaces from the dataset. Since illumination is usually the cause behind the largest variations in face images, the first three eigenfaces will mainly capture the information of 3-dimensional lighting changes, which has little contribution to face recognition. By discarding those three eigenfaces, there will be a decent amount of boost in accuracy of face recognition, but other methods such as fisherface and linear space still have the advantage.

See also

Craniofacial anthropometry

Human appearance

Pattern recognition

References

Further reading

M. Kirby; L. Sirovich (1990). "Application of the Karhunen-Loeve procedure for the characterization of human faces". *IEEE Transactions on Pattern Analysis and Machine Intelligence*. 12 (1): 103–108. doi:10.1109/34.41390.

A. Pentland, B. Moghaddam, T. Starner, O. Oliyide, and M. Turk. (1993). "View-based and modular Eigenspaces for face recognition". Technical Report 245, M.I.T Media Lab.

M. H. Yang (2000). "Face recognition using kernel eigenfaces". *Proceedings International Conference on Image Processing*. Vol. 1. pp. 37–40. doi:10.1109/ICIP.2000.900886.

R. Cendrillon; B. Lovell (2000). "Real-time face recognition using eigenfaces" (PDF). *Visual Communications and Image Processing*. pp. 269–276. doi:10.1117/12.386642.

T. Heseltine, N. Pears, J. Austin, Z. Chen (2003). "Face Recognition: A Comparison of Appearance-Based Approaches". *Proc. VIIth Digital Image Computing: Techniques and Applications*, vol 1. 59–68.

D. Pissarenko (2003). Eigenface-based facial recognition.

F. Tsalakanidou; D. Tzovaras; M.G. Strintzisa (2003). "Use of depth and colour eigenfaces for face recognition". *Pattern Recognition Letters*. 24 (9): 1427–1435. Bibcode:2003PaReL..24.1427T. doi:10.1016/S0167-8655(02)00383-5.

Delac, K., Grgic, M., Liatsis, P. (2005). "Appearance-based Statistical Methods for Face Recognition". *Proceedings of the 47th International Symposium ELMAR-2005 focused on Multimedia Systems and Applications, Zadar, Croatia, 08-10 June 2005*, pp. 151–158

External links

[Face Recognition Homepage](#)

[PCA on the FERET Dataset](#)

[Developing Intelligence Eigenfaces and the Fusiform Face Area](#)

[A Tutorial on Face Recognition Using Eigenfaces and Distance Classifiers](#)

[Matlab example code for eigenfaces](#)

[OpenCV + C++Builder6 implementation of PCA](#)

[Java applet demonstration of eigenfaces Archived 2011-11-01 at the Wayback Machine](#)

[Introduction to eigenfaces](#)

[Face Recognition Function in OpenCV](#)

[Eigenface-based Facial Expression Recognition in Matlab Archived 2017-06-04 at the Wayback Machine](#)